Company Intelligence Platform — Interview Explanation Structure

Use this as your talk-track when the interviewer says “explain me your project.” Speak in this order: Problem → Solution → Technical Evolution (Phase 1 → 5) → Tech Stack → Anticipated Follow-ups.

---

1. The Problem Statement (The “Why”)

“Universities, placement offices, and enterprise recruitment agencies struggle to collect, validate, and query deep, structured corporate profiles for hundreds of target companies. The data is either too shallow or too scattered. Students need to know more than just a company name — they need to know the tech stack, specific hiring roles, future AI initiatives, stock performance, and cultural fit.”

Three core pain points:

1. **High Data Fragmentation & LLM Hallucination Risks** Relying on manual web research or a single AI prompt yields outdated metrics, missing attributes, and frequent LLM hallucinations (e.g., inaccurate financial figures, fictitious tech stacks, or incorrect Glassdoor ratings).

2. **Lack of Automated Data Quality Control (QA)** Raw AI outputs frequently contain broken URLs, invalid email syntaxes, missing mandatory fields, and out-of-range ratings (e.g., a rating of 12.5/5). Manually inspecting thousands of incoming parameters requires endless manual script writing.

3. **Inability to Perform Contextual & Semantic Querying** Traditional relational databases rely on exact keyword matches. Recruiters and students cannot ask natural-language conceptual queries like: *“Show companies scaling their AI teams that test dynamic programming in round 2 and offer work-from-home options.”*

The Challenge (scale statement): “Manually researching 1,100+ companies across 163 different parameters (like AI adoption, project roadmaps, and financial health) is humanly impossible to keep updated. Standard scrapers fail because the data is unstructured and inconsistent.”

Your personal scope: You did this for 116 companies that came to Anna University in previous placement years, mapped across 163 “golden parameters” you explored and finalized — e.g. `tech_stack`, `hiring_velocity`, `yoy_growth_rate`, `ai_ml_adoption_level`, `interview_competencies`, `hiring_rounds`, etc.

---

2. The Solution (The “What”)

“I built an Enterprise-Grade Company Intelligence Platform. It’s a full-stack system that uses Agentic AI to autonomously research, consolidate, and validate corporate data. It transforms raw, scattered web data into a ‘Golden Record’ for each company, accessible via a RAG-powered chatbot and a semantic search engine.”

In plain terms it provides: what roles a company hires for, how their evaluation/interview process works, what topics come up in online assessments, and more — all backed by verified, structured data.

---

3. The Technical Evolution (The “How”)

Phase 1 — Foundation: Schema & Database Automation

- Established the core 163-parameter research specification to profile target companies (e.g., Blinkit).
- Designed the initial database DDL schemas using **Supabase (PostgreSQL)**, implementing triggers and stored procedures.
- When raw data hits the staging table, **database triggers parse delimited text strings**, perform **dynamic FK lookups** for cities and countries, and populate the normalized tables — all inside **atomic SQL transactions**.
- Built the initial stack: **Next.js + FastAPI**. Started by gathering data using LLMs (GPT, Gemini) directly and storing it in Supabase — but realized a single LLM often hallucinates or gives incomplete data across 163 parameters.

Phase 1.5 — QA Automation Framework (Pytest)

- Built a programmatic **QA engine** to ensure every incoming record strictly conforms to data type, format, and business logic constraints — covering granularity, data volatility, nullability, regex patterns, and boundary limits.
- Since testing every possible combination is impractical, developed **65 individual Python test scripts, combined into one unit test suite**, to cover all cases efficiently (see the math breakdown in the Q&A section below).

Phase 2 — The Agentic Shift (Automation & Precision)

- To solve the data quality issue, implemented an **Agentic Workflow using LangGraph**.
- Instead of a single prompt, used a **Multi-Agent Consensus model**: separate agents (**Claude, Gemini, Llama**) independently research the same company.
- A **Consolidator Agent** merges the best data using a weighted scoring model:
  1. **Data Freshness (35% weight)** — compares timestamps of source documents retrieved by each LLM; newer financial reports score higher.
  2. **Research Accuracy (45% weight)** — evaluates source reliability; primary sources like SEC filings or official press

releases are weighted higher than third-party blogs.
3. **Rule Compliance (20% weight)** — checks strict type safety, ranges, and schema compliance.

- A **Validator Agent** (Gemini) checks the consolidated data against a strict schema — acting purely as an **in-context auditor**, relying only on the evidence provided in the prompt, not on its own trained/pre-training knowledge (to avoid hallucination/outdated knowledge).
- If data is wrong, it triggers a **self-healing loop** to re-research until it's correct, with a **fallback model** triggered if the primary model fails.
- Since generating all 163 parameters in one pass is difficult, the process is **batched — 33 parameters per batch**.

Phase 5 — Deterministic Validation (No LLM)

- Phase 5 uses **no LLM at all** — it's 100% deterministic code built in **Python using Pydantic validation rules**.
- It programmatically validates schema types, boundary conditions, and domain rules across all 163 fields.
- If Phase 5 passes → writes directly to Supabase.
- If a specific field fails → Phase 5 **isolates that field ID** and passes only that field into **Phase 7's regeneration loop**, avoiding expensive full-company re-runs.

Phase 3 — Production, RAG & DevOps

- Added a **RAG (Retrieval-Augmented Generation) chatbot**: users can ask natural questions like *"Which companies are moving into Generative AI in the healthcare sector?"* and the system performs a **semantic search across the vector database** to give an intelligent answer.
- **Containerization & CI/CD:** Containerized frontend and backend microservices. Built a **Jenkins CI/CD pipeline** automating Checkout → Lint → Build → Test → Docker Build & Push → Deploy.

---

4. Tech Stack

Layer	Technology
Frontend	Next.js, Tailwind CSS
Backend	FastAPI (Python), LangGraph (Agent Orchestration)
AI	Gemini, Claude, Llama 3 (via Groq/OpenRouter), LangChain
Database	Supabase (PostgreSQL), SQLite / Pgvector (Vector Store)
DevOps	Docker, Nginx (Reverse Proxy), Jenkins (CI/CD)

---

5. Anticipated Follow-Up Questions (with your answers)

Q: How do you handle updating the data?

“It’s admin-triggered. When the admin clicks the update button, the agentic pipeline kicks off — the multi-agent research runs again, the Consolidator and Validator agents re-check the data, and the freshly researched/updated data is placed back into Supabase.”

Q: How did you arrive at exactly 65 test cases (not 163 × 18)?

This is your strongest engineering-judgment story — walk through it step by step:

- Naively: 163 parameters × 18 test rules (Nullability, Min/Max Length, Regex Format, Emoji Injection, Negative Numbers, Range Boundaries, Special Characters, Datetime format, etc.) = **2,934 test cases**.
- “That would be impossible to manage — it’s bad software engineering, creating duplicated, unmaintainable code.”
- **The fix:** use `@pytest.mark.parametrize` to group similar parameters and test rules into a single file, instead of writing one file per test case.

Concrete example — `tc_3.1.py` (URL Validation): - In `test_Cases.xlsx`, there are 8 URL-type parameters (`website_url`, `linkedin_url`, `facebook_url`, `instagram_url`, `twitter_handle`, `logo_url`, `ceo_linkedin_url`, `marketing_video_url`). - Each of those 8 parameters has 5 test cases in Excel (Valid URL, Missing `https://`, Trailing Space, Malformed Domain, Null Check). - 8 parameters × 5 test cases = **40 Excel test case rows**. - Instead of writing 40 separate `.py` files, **all 40 test cases live inside ONE file (`tc_3.1.py`)** using `@pytest.mark.parametrize`. - Doing this grouping across all parameter types is how the 2,934 theoretical cases were consolidated down to **65 maintainable, parametrized test files**.

Q: How does the Semantic Engine work?

(Flag: your source notes end here — this section needs your own elaboration before the interview. At minimum, be ready to describe: data → embeddings → vector store (Pgvector) → similarity search → RAG chatbot retrieves relevant company records → LLM generates the natural-language answer grounded in retrieved records.)

Quick Delivery Tips

- Lead with the **problem’s scale** (1,100+ companies, 163 parameters) before your solution — it justifies why you needed agentic automation instead of a single script.
- The **65 vs. 2,934 test case** story is a great signal of engineering maturity — mention it even if not asked, it shows you think about maintainability.
- Have one crisp sentence ready for **why multi-agent consensus over single-LLM**: hallucination reduction via cross-verification + weighted scoring (freshness/accuracy/compliance).
- Be ready to explain the **Phase 5 → Phase 7 loop** clearly: deterministic Python validation, not an LLM, is what actually gates writes to the database — this shows you understand where determinism belongs vs. where AI belongs.

---

6. Possible Interviewer Questions

Architecture / System Design

1. Walk me through the end-to-end data flow, from a company name to a “Golden Record” in the database.
2. Why did you choose Supabase (PostgreSQL) over a NoSQL database like MongoDB for 163 semi-structured parameters?
3. Why triggers and stored procedures at the database level instead of handling FK lookups and parsing in the application layer (FastAPI)?
4. What does “atomic SQL transaction” mean in your context, and why did it matter here? What happens if a transaction fails midway?
5. How is your staging table different from your normalized tables? Why not write directly to the normalized tables?
6. How does your system scale if you go from 116 companies to 1,100+? What would break first?
7. Why FastAPI over Flask/Django for the backend? Why Next.js over plain React for the frontend?
8. Where does the Nginx reverse proxy sit in your architecture, and why did you need it?

Agentic AI / LangGraph / Multi-Agent

9. Why LangGraph specifically — why not just orchestrate multiple API calls yourself in Python?
10. Explain the Consolidator Agent’s weighted scoring (35/45/20) — why those specific weights? Could they be tuned?
11. How does the Consolidator Agent actually compare “data freshness” across sources — what signals does it use?
12. What is an “in-context auditor,” and why does it matter that the Validator Agent doesn’t rely on trained/pretrained knowledge?
13. What happens if all agents disagree significantly? Is there a threshold, or does it always pick the highest-scoring one?
14. What’s a “self-healing loop”? How many times can it retry before giving up, and what happens on repeated failure?
15. What is the fallback model, and when exactly does it trigger — timeout, API error, low confidence score?
16. Why batch the 163 parameters into groups of 33? Why that number and not some other batch size?
17. How do you prevent the multiple agents (Claude, Gemini, Llama) from being expensive to run for every company, every update?
18. Could two agents hallucinate the same wrong fact and still “agree”? How do you guard against that?

Data Quality / Testing

19. Walk me through your QA framework — how do the 18 test rules map onto the 163 parameters?
20. Explain the math: how did you go from a theoretical 2,934 test cases down to 65 maintainable files?
21. What is `@pytest.mark.parametrize` and why does it solve the duplication problem?
22. Give me another example (besides URL validation) of a parameter type and how you’d write its parametrized test file.

23. What's the difference between your Pytest QA layer (Phase 1.5) and your Phase 5 deterministic Pydantic validation? Aren't they redundant?
24. Why Pydantic for Phase 5 instead of just more Pytest checks?
25. If a field fails Phase 5, how exactly is it isolated and routed to Phase 7 without re-running the whole company's research?
26. What's an example of a "boundary condition" or "domain rule" you validate (e.g., glassdoor rating range, regex for emails/URLs)?

RAG / Semantic Search

27. Explain your RAG pipeline in detail — embedding model used, chunking strategy, vector store, retrieval + generation flow.
28. Why Pgvector/SQLite for vectors instead of a dedicated vector DB like Pinecone or Weaviate?
29. How do you keep the vector index in sync when the "Golden Record" gets updated by the admin?
30. How would the chatbot answer a query like "companies scaling AI teams testing dynamic programming in round 2 with WFH options" — walk through retrieval to answer, step by step.
31. How do you evaluate/measure the quality of RAG answers (hallucination checks, relevance)?

DevOps

32. Walk me through your Jenkins pipeline stages in detail — what does each stage actually check?
33. What do you containerize separately — is it one Docker image or multiple microservices?
34. What happens if a build fails at the "Test" stage — does it block deployment?
35. How do you handle secrets/API keys (LLM API keys, Supabase keys) in your CI/CD pipeline?

Product / Business Sense

36. Who is the end user — students, placement cells, or recruiters — and how does the UI differ for each?
37. How do you keep data legally/ethically sound when scraping/researching company information?
38. What was the single hardest technical problem you solved in this project, and why?
39. If you had 2 more weeks, what would you add or fix?
40. What would you do differently if you rebuilt this from scratch today?

Follow-ups You Already Prepared For

41. How do you handle updating the data when new information is needed? *(admin-triggered → agentic pipeline reruns)*
42. How did you arrive at exactly 65 test cases instead of 163 × 18? *(parametrize consolidation — see Section 5)*
43. How does the semantic search/chatbot actually work end-to-end? *(finalize your own answer here — this is currently your weakest-documented area)*

Gap to Close Before the Interview

Your source notes don’t fully specify the **Semantic Engine’s internals** (embedding model, chunking approach, exact vector store config, retrieval-to-generation wiring). Given Q27–Q31 above are very likely to come up, it’s worth nailing down concrete answers to those before the interview.